

A 3D rendering of a warehouse conveyor belt system. Several cardboard boxes are moving along the belt. Red laser lines form a grid pattern on the floor and around the boxes, suggesting a detection or tracking system. The scene is brightly lit, with a focus on the boxes and the laser grid.

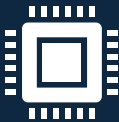
Literature Review: TinyML-Based Smart Intrusion Detection System with ESP32

BSCE22004
BSCE22025
BSCE22048

Key Enabling Technologies



Model Compression & Quantization Converts 32-bit weights to 8-bit (int8), reducing model size by ~75% while retaining ~97% accuracy.

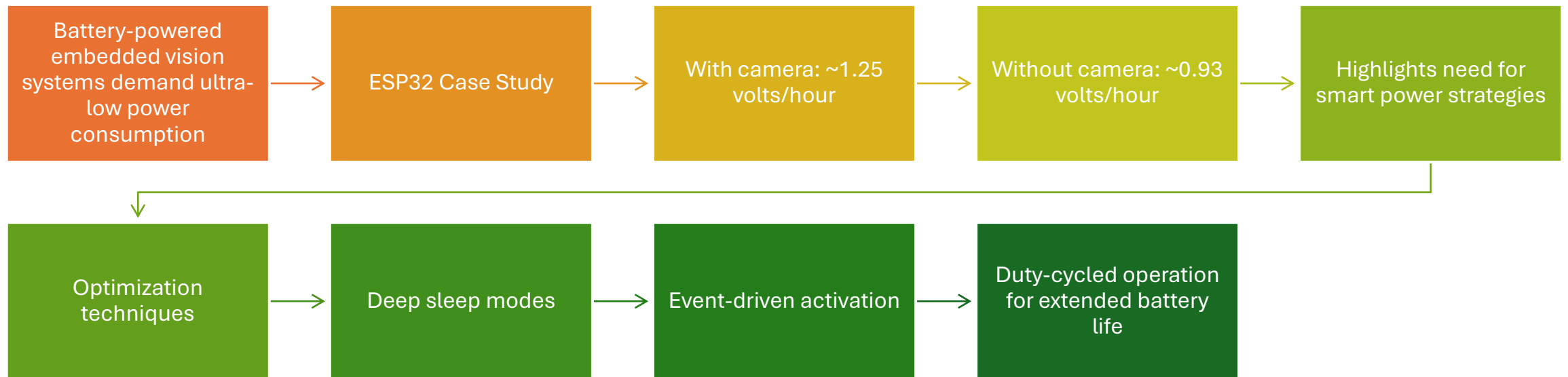


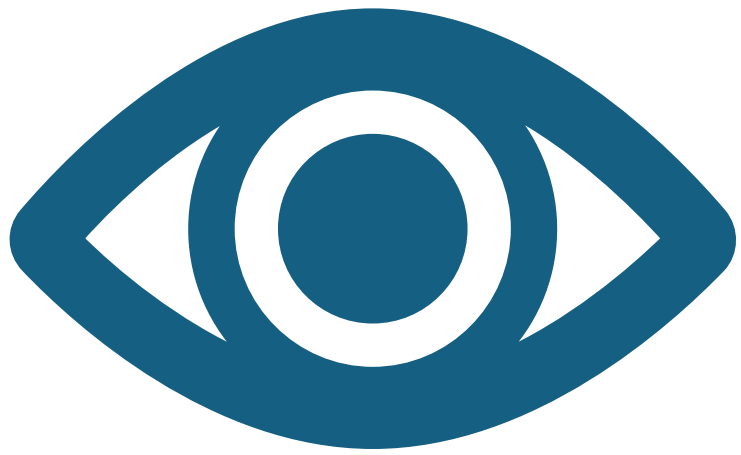
TensorFlow Lite for Microcontrollers (TFLite Micro) Lightweight, OS-free ML framework for ARM Cortex-M, ESP32, and DSPs; highly portable across platforms.



Edge Impulse Platform End-to-end TinyML development suite with EON Compiler for optimized deployment on microcontrollers like ESP32 and Arduino.

Energy Efficiency Considerations





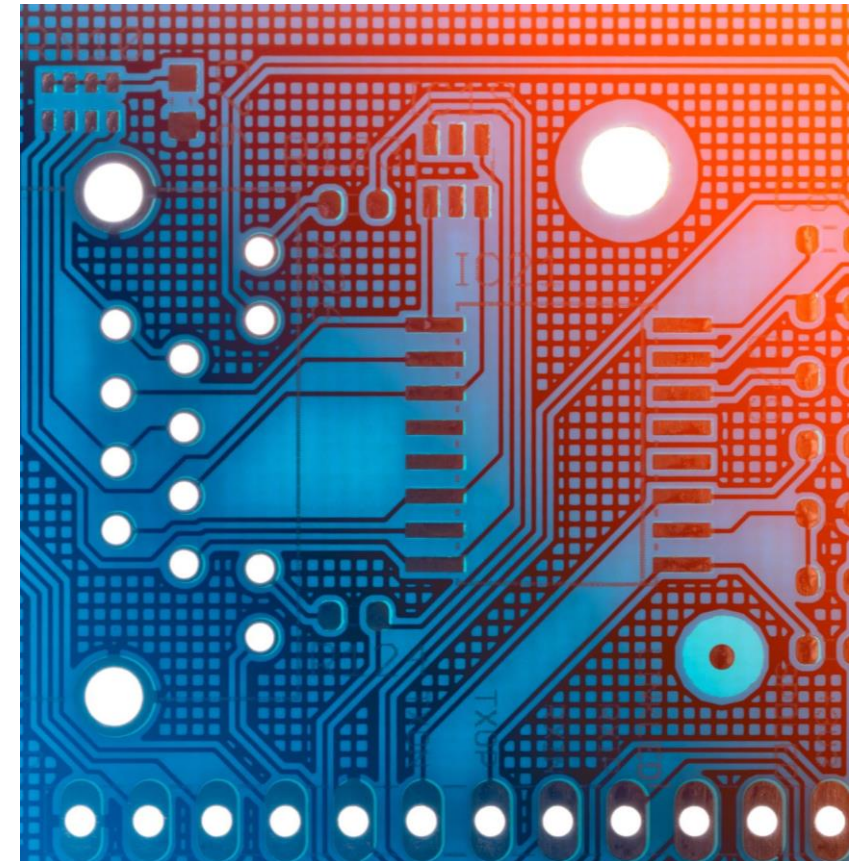
ESP32 Platform for Embedded Vision

ES32 ARCHITECTURE

- Why esp32?
Combines processing power, wireless connectivity, and affordability — ideal for embedded ML.
- Core specs
- Dual-core 32-bit processor @ 240 MHz
- 520 KB SRAM + up to 16 MB external flash
- Parallel processing: one core for networking, one for inference

ESP32-CAM Variant

- Integrated OV2640 camera (2 MP)
- Supports UXGA (1600×1200), SVGA (800×600), and TinyML-optimized 96×96 resolution
- Tailored for vision-based ML tasks



Computational Capabilities and Limitations (ESP32)

Processing Bottlenecks

Complex vision models overwhelm ESP32, causing low frame rates (1–3 FPS) and inference delays.

Memory Constraints

Limited SRAM restricts neural network size and complexity

Camera Initialization Issues

Frequent errors: “camera init failed,” brownout detection, and frame buffer timeouts



Successful Implementation Strategies (ESP32 Vision)

Resolution Optimization Use low-resolution inputs (e.g., 96×96 or 128×128 pixels) to reduce computational load while preserving detection accuracy.

Example: Visual Wake Words dataset uses 96×96 grayscale images for person detection.

Model Architecture Selection Choose lightweight models like MobileNet-SSD or FOMO for resource-constrained devices.

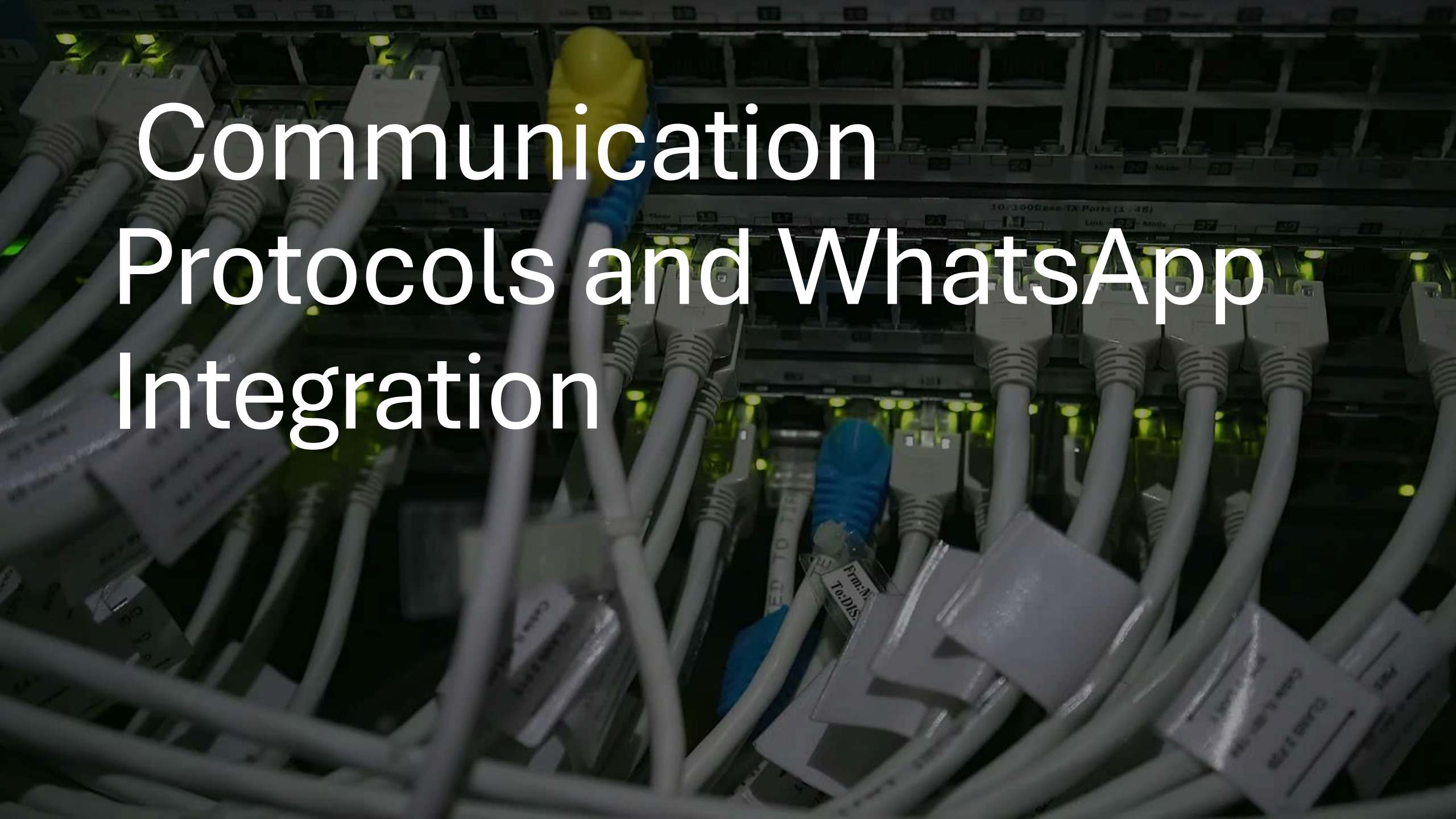
MobileNet uses depthwise separable convolutions to balance efficiency and feature extraction.

Hybrid Edge-Cloud Approaches Combine local inference with cloud-based verification for improved accuracy.

Example: EagleEye project uses local processing with cloud-assisted labeling.

Mobile Architecture

- **MobileNet & MobileNet-SSD:** Efficient use of depthwise separable convolutions makes MobileNet ideal for embedded vision. Used with SSD for real-time detection, achieving over 99% precision for vehicles and 97–99% for person detection in controlled settings. Performance drops in variable lighting and occlusion conditions.
- **FOMO (Faster Objects, More Objects):** Designed for resource-constrained devices, FOMO predicts object centers rather than bounding boxes, reducing computational cost. Microcontroller tests show F1 scores of 58–95% with inference latency below 1 ms.
- **TinyYOLO & YOLOv5/v8:** YOLO frameworks provide excellent detection, but require careful optimization for microcontrollers. TinysissimoYOLO achieves efficient inference (~3 FPS) with low error counts on edge devices. TinyML studies show FOMO is generally more efficient on constrained platforms

A close-up photograph of a network switch or patch panel. Numerous white Ethernet cables are plugged into the ports. Some cables have blue or yellow RJ45 connectors. Several cables have white labels attached to them, some with handwritten text like 'From: N' and 'To: DIS'. The background is dark, and the overall image has a slightly blurred, high-contrast aesthetic.

Communication Protocols and WhatsApp Integration

Messaging API Integration for ESP32

CallMeBot API

Function: Provides a straightforward HTTP-based API for sending WhatsApp messages from embedded devices.

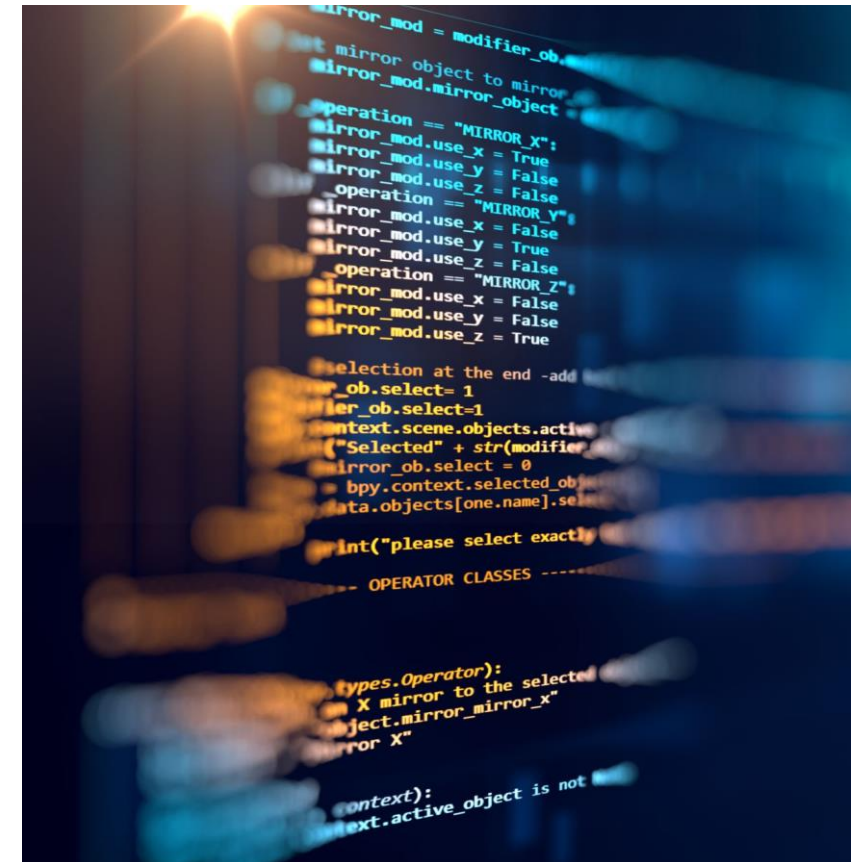
Setup: Requires registration of the target phone number and generating an API key.

Mechanism: ESP32 uses HTTP POST requests with URL-encoded content.

Capabilities: Supports text and basic alert transmission.

⚠ Limitations & Alternatives

- CallMeBot imposes **rate limits** and may have **reliability issues** during high-volume usage
- Image Processing not supported





Alternatives

Telegram Bot API

- Offers more **robust programmatic access**.
- Supports **image transmission** alongside text.

MQTT Publish-Subscribe

- Enables **decoupled communication** between IoT devices and notification services.

SMS via GSM Modules

- Provides notification **independent of internet connectivity** (but at a higher cost).

A 3D rendering of a warehouse conveyor belt system. Several cardboard boxes are shown on the belt, moving towards the viewer. The boxes have various labels, including one that says "FRAGILE". A red laser grid is overlaid on the scene, consisting of a grid of lines and small crosshairs, suggesting a real-time detection or tracking system. The background is a bright, white, slightly blurred area, possibly representing a warehouse floor or ceiling.

Real-Time Detection and Response Systems

System Architecture Design

Sensor Fusion for Power Efficiency:

Low-power motion sensors (PIR) trigger the camera only when needed, significantly extending battery life.

On-Device Processing Pipeline:

Trigger → Capture → Process → Detect → Alert

A TinyML model performs person detection directly on the device.

Edge AI for Privacy and Speed:

Processing data locally prevents the transmission of sensitive visual information and allows for fast, intelligent decision-making.

Performance Metrics

Detection Accuracy

The fundamental measure of model effectiveness.

Real-world performance typically ranges from **85-95%**, depending on environmental conditions [43].

Latency

Time from event occurrence to notification delivery.

Emergency Systems: Aim for < 500 milliseconds [40].

Doorway Monitoring: 2-5 second delays are generally acceptable.

False Positive Rate

Excessive false alarms reduce user trust and utility.

Target: Achieve rates **below 5%** in controlled scenarios [26].

Energy Consumption

Critical for battery-powered deployments.

Duty-cycled operation (using motion triggers) can extend battery life from days to **months**.

Environmental Robustness

Environmental Robustness

Real-world deployments must overcome challenges not found in labs:

Lighting Variations 💡

- **Challenge:** Accuracy degrades significantly in low-light and nighttime conditions.
- **Solution:** Combine image enhancement algorithms (e.g., CLAHE) with suitable neural networks (e.g., VGG16) to improve accuracy (up to 92%) [6, 43].

Occlusions and Partial Views 🚪

- **Challenge:** Doorways often provide only partial body visibility.
- **Solution:** Training datasets **must** include examples of partial occlusions to ensure robust detection [26].

Weather Conditions 🌧️

- **Challenge:** Rain, fog, and precipitation can degrade image quality in outdoor deployments.
- **Solution:** Requires hardware considerations like weatherproofing and lens protection.



Related Systems and Comparative Analysis

Limitations of Commercial Smart Doorbells

While comprehensive, commercial systems present several drawbacks addressed by edge-based approaches:

Subscription Requirements 💳

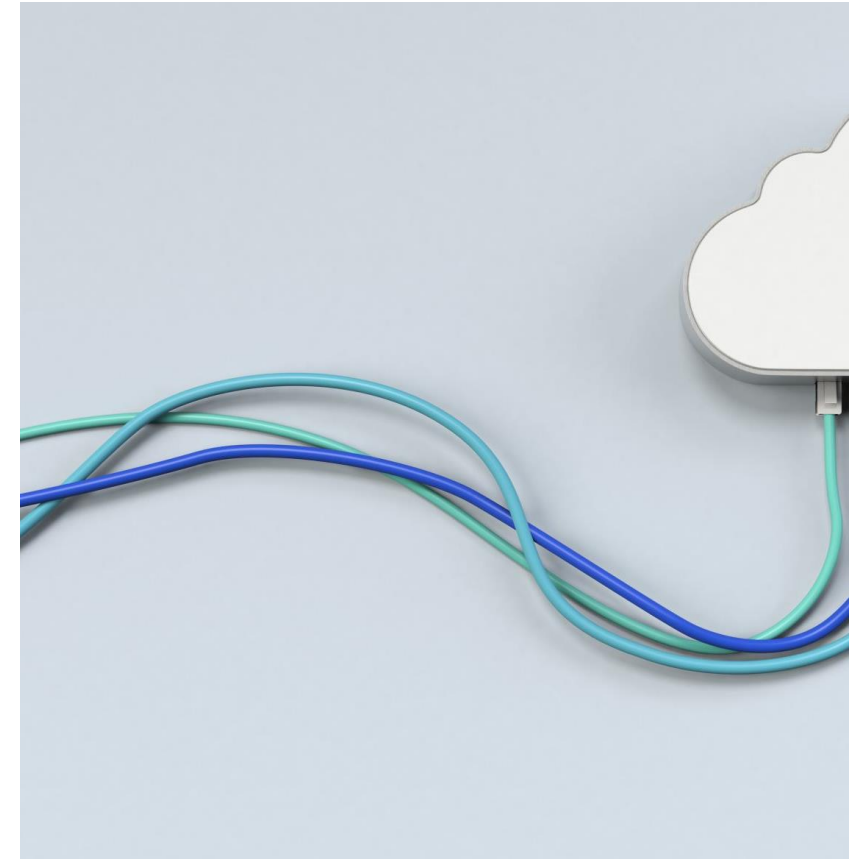
- Cloud storage and advanced features often require ongoing subscription fees, increasing the total cost of ownership [44].

Privacy Concerns 🔒

- Transmitting video data to cloud servers raises privacy issues, heightened by incidents of unauthorized access and data breaches [45].

Connectivity Dependence 🚫

- Cloud-reliant systems become non-functional during internet outages, eliminating security coverage when it might be most needed [45]



Related Technologies and Research Areas

Face Recognition Door Lock Systems

- Offers enhanced security through biometric authentication [41, 44].

Drawback: Introduces more significant privacy concerns and higher computational requirements compared to simple person detection.

•IoT-Based Smart Doorbell (Prototypes)

- Research prototypes (e.g., ESP32, Raspberry Pi) validate the feasibility of the technical approach [44, 45].

Limitation: Often lack optimization for production deployment.

Privacy-Preserving Surveillance

- Research demonstrates techniques to detect human presence and activity *without* capturing identifiable visual information [45].

Methods: Includes skeletal tracking, silhouette detection, or encrypted visual features.



Comparative Advantages of the EagleEye Approach

The EagleEye system distinguishes itself through several key design choices:

Local Processing

- Complete person detection inference executes **on-device** [1, 2].
- Eliminates cloud dependence and protects user privacy.

Cost Effectiveness

- Utilizes affordable commodity hardware (e.g., ESP32-CAM @ **\$10-15**) [46].
- Avoids high initial costs (\$100-\$200) and recurring subscriptions.

Minimal Latency

- Edge processing eliminates network round-trip delays.
- Enables **sub-second** response times.

Independence from Internet

- Core detection functionality operates with intermittent or no connectivity.
- Network access is only required for sending notifications.

Challenges and Limitations

A thick, hand-drawn style orange line underlining the text.

Technical Constraints

Limitations and Trade-offs

Computational Limitations 🕒

- **Platform:** The ESP32 imposes strict constraints on model complexity and speed.
- **Performance:** Achieves **1-3 FPS** for person detection, which is acceptable for doorway monitoring but not for continuous video surveillance [15, 18, 20, 21, 25].

Model Size Restrictions 💾

- **Flash:** TinyML models must fit within approximately **400-500 KB** of flash memory.
- **RAM:** Must use less than **200 KB of RAM** during inference.
- **Solution:** Requires careful architecture selection and aggressive model compression.

Accuracy-Efficiency Trade-offs ⚖️

- Reducing model complexity to meet resource constraints inevitably impacts detection accuracy.
- TinyML models are **more sensitive to dataset quality** than larger models, requiring careful dataset curation [26].

Conclusion



The New Opportunity

- **The Convergence:** TinyML, edge computing, and affordable embedded hardware create new opportunities for intelligent, privacy-preserving security systems.
- **The Approach:** ESP32-based person detection with **local processing** is a technically feasible approach that solves key problems of cloud-dependent systems.
- **Key Finding 1: Technical Feasibility**
 - Research shows optimized neural networks can run on ESP32 hardware at acceptable frame rates (1-3 FPS) and accuracy (85-95%) [2, 15, 26].
- **Key Finding 2: Energy Viability**
 - TinyML offers dramatic energy efficiency, enabling extended battery operation when paired with smart power management [6, 35].

Key Advantages & Challenges

Local processing fundamentally addresses privacy concerns by eliminating the need to send visual data to external servers [2, 34].

Key Finding 4: Cost Effectiveness

- Using commodity hardware (e.g., ESP32-CAM @ \$10-15) makes intelligent monitoring accessible, avoiding high upfront costs and subscriptions [8].

Key Finding 5: Integration Challenges

- Success requires careful model optimization, power management, and environmental robustness.
- The ESP32 platform imposes real constraints, forcing **trade-offs** between capability and resource use.